AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

Dept. of Computer Science
Faculty of Science and Technology

CSC2210: OBJECT ORIENTED PROGRAMMING 2

Fall 2025-2026
Section: [G]
Group No: 09

Project Report On

Project name [Farm Invoice Manager]

Supervised By
Md. Hasibul Hasan

Submitted By: MAHMMOD, MD. SHAFI

Name	ID
TASNIM, JARIN	**23-50749-1**
MAHMMOD, MD. SHAFI	**23-54122-3**
RISHAD, NAJIAT ISLAM	**23-55218-3**

Obtained Marks for CO2 and CO3 (Description given in the following page)

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria (CO2)	**Total =**	**Evaluation Criteria (CO3)**		**Total =**	
Requirement fulfillment		Organization of the application			
Validation		Representation and Integration of Database			
Verification		Graphical User Interface			

CO2: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.					
Assessment Criteria	**Not Attended/ Incorrect (0)**	**Inadequate (1-2)**	**Average (3)**	**Good (4)**	**Excellent (5)**
Evaluation Criteria	**Evaluation Definition**				
Requirement fulfillment	Fails to demonstrate any understanding of real-life scenario-based project development or functional requirement identification. There is no attempt to depict a project or identify functional requirements accurately.	Demonstrates limited understanding of real-life scenario-based project development and functional requirement identification. The project depicted lacks coherence or relevance to real-life scenarios, and functional requirements are inaccurately identified or insufficiently described.	Presents a basic depiction of a real-life scenario-based project and identifies some functional requirements. However, the project lacks depth or complexity, and some functional requirements may be vaguely defined or missing key details.	Effectively demonstrates a realistic scenario-based project and accurately identifies most functional requirements. The project is well-developed with appropriate complexity, and functional requirements are clearly articulated with relevant details.	Exhibits an exceptional understanding of real-life scenario-based project development and accurately identifies all functional requirements. The project is meticulously developed with thorough attention to detail, reflecting a comprehensive understanding of Object-Oriented Programming project development activities.
Validation	Fails to demonstrate any understanding or implementation of validation forms in their system. There is no attempt to deal with data validation, and validation requirements are completely ignored or incorrectly applied.	Demonstrates limited understanding of validation forms and data validation techniques. While some attempt may be made to implement validation, it is incomplete or poorly executed, leading to inadequate handling of data validation.	Shows a basic understanding of validation forms and data validation techniques. They attempt to implement validation, but some aspects may be missing or incorrectly implemented, resulting in partial or inconsistent handling of data validation.	Effectively demonstrates the use of validation forms and implements data validation techniques. Validation is mostly accurate and comprehensive, ensuring the proper handling of data input and verification in the system.	Exhibits an exceptional understanding and implementation of validation forms and data validation techniques. Validation is meticulously implemented with thorough attention to detail, ensuring robust data validation procedures and contributing to the overall reliability and integrity of the system.

CO2: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.					
Assessment Criteria	**Not Attended/ Incorrect (0)**	**Inadequate (1-2)**	**Average (3)**	**Good (4)**	**Excellent (5)**
Evaluation Criteria	**Evaluation Definition**				
Verification	Fails to demonstrate any attempt to verify the system data or functional requirements. There is no evidence of understanding or implementation of verification processes, and data flow is not considered.	Demonstrates limited understanding of verification processes and data flow in the system. Verification attempts are incomplete or inaccurate, and there is insufficient consideration given to ensuring data integrity and functionality.	Shows a basic understanding of verification processes and attempts to verify system data. However, verification efforts may be inconsistent or lack thoroughness, and there may be gaps in ensuring proper functional requirements and data flow.	Identifies and verifies system data, ensuring proper functional requirements are met. Verification efforts are mostly accurate and thorough, with attention to ensuring data integrity and appropriate data flow within the system.	Exhibits an exceptional understanding of verification processes and meticulously verifies system data. Verification efforts are comprehensive and precise, with a keen focus on ensuring all functional requirements are met and maintaining proper data flow throughout the system.

CO3: Prepare and Explain a real life desktop based application synthesizing several component of C# along with development tools to adhere the given requirements.					
Assessment Criteria	**Not Attended/ Incorrect (0)**	**Inadequate (1-2)**	**Average (3)**	**Good (4)**	**Excellent (5)**
Evaluation Criteria	**Evaluation Definition**				
Organization of the application	Fails to identify any suitable real time application or requirements for project development activities related to OOP.	Limited understanding about the project scopes and scenarios or identification of functional requirements.	Lacks depth or relevance to OOP project development activities and may contain inaccuracies. Real-life scenarios are mentioned, but the discussion lacks depth or clarity.	Consider and integrate the idea of several core aspects of the project along with relevance to real-life scenarios. Demonstrating a solid understanding of the application presentation.	Generalize and exhibits an exceptional understanding of project preparation according to real-life scenarios. Also contains proper and insightful identification of the system which is comprehensive and precise.

CO3: Prepare and Explain a real life desktop based application synthesizing several component of C# along with development tools to adhere the given requirements.					
Assessment Criteria	**Not Attended/ Incorrect (0)**	**Inadequate (1-2)**	**Average (3)**	**Good (4)**	**Excellent (5)**
Evaluation Criteria	**Evaluation Definition**				
Representation and Integration of Database	Fails to identify and present any understanding or implementation of database. Also failed to integrate the data with the project itself.	Limited understanding of the database concepts or their proper way of using in a real time project. While some attempt may be made to implement but it is incomplete or poorly executed, leading to inadequate design.	Lacks depth or relevance to database integration with the application. Shows a basic understanding but some aspects may be missing or incorrectly implemented, resulting in partial or inconsistency. May lack proper normalization.	Integrate the database with the forms properly and implements it with proper validation which is mostly accurate and comprehensive, ensuring the proper handling of data input and verification along with general normalization.	Exhibits an exceptional understanding and implementation of database ensuring attention to detail, and robust data manipulation procedures and contributing to the overall clarity.
Graphical User Interface	Fails to present or prepare GUI based application interfaces. There is no evidence of creating or integrating such things according to their usefulness.	Limited understanding of graphical user interfaces. Lack of design knowledge. Very poor attempt to make such things which are currently obsolete or can't be identified as coherent.	Shows a basic understanding of creating user interfaces. Most of them are interconnected but maybe some of them lack it. However, most of it can be described as user friendly.	Effectively identifies and meet the consider the simplicity. Design related works are mostly accurate and taken proper attention to ensuring a user-friendly coherent system.	Exhibits an exceptional work design following a high standard of simple and elegant work. Several controls and mechanism has been organized in a preferred way according to the coherent usage.

Table of Contents: **Page no**

1. Chapter 01 (Introduction) 4
2. Chapter 02 (User Story) 4
3. Chapter 03a (ER Diagram) 5
4. Chapter 03b (SQL Queries) 6
5. Chapter 04 (Screenshots) 11

Chapter 01: Introduction

The **Farm Invoice Manager – Billing Management System** is a desktop-based application developed using **C# Windows Forms** and **SQL Server** to manage billing, sales, products, customers and users in a farm-based business. The main goal of the system is to simplify and automate the manual invoice and sales management process, reducing errors and improving productivity.

The system provides a **secure login** with role-based access for **Admin** and **Employee**. Admin users can manage users, products and customers and view sales reports, while employees can make sales, view customers and see their own information. The application supports **product management**, **customer management**, **invoice generation**, **sales reporting** and **printable invoices**. Features such as **password recovery**, **username and password reset**, **premium membership**, **date-wise sales reports** and **graphical sales analysis** improve usability and decision-making.

All input fields are checked while the user types, all database queries use parameters, and saving an invoice (invoice details, items and stock update) is done in one transaction, so the data stays correct. With a user-friendly interface and structured data handling, the Farm Invoice Manager helps farm businesses keep accurate records, track inventory, manage customers and produce reliable financial reports. Overall, this project shows the use of object-oriented programming, database connectivity and real-world business logic in a desktop application.

Chapter 02: User Story

The Farm Invoice Manager – Billing Management System is designed for two user roles: **Admin** and **Employee**. Each role has its own responsibilities and access permissions to keep farm billing and sales safe and efficient. Below is a short overview of how each user works with the system.

As an Admin

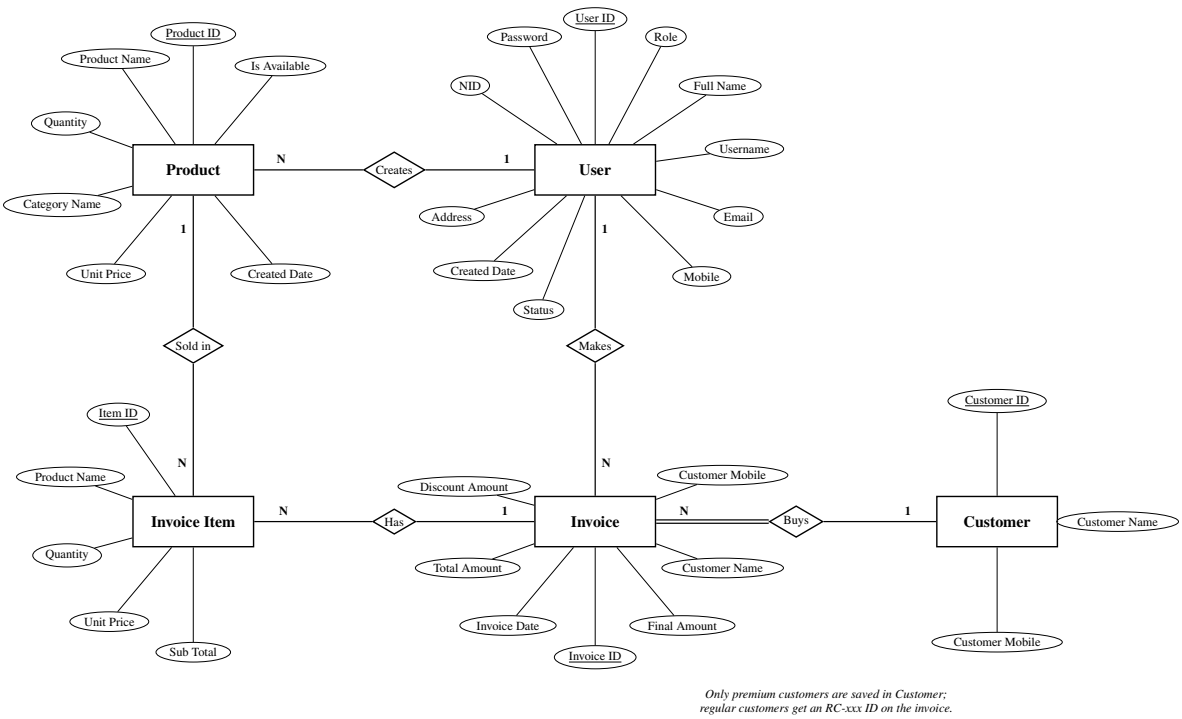
- Log in securely to the admin dashboard using a username (or User ID) and password.
- Add, update, search or remove system users and manage their personal and login information.
- Add new farm products, update product details such as name, category, stock and unit price, and remove products that are not used in any invoice.
- View, search and delete premium customers.

- Create sales invoices with a discount and add new premium customers during a sale.
- View, print and delete invoices.
- View sales reports with date-wise filtering, customer search and a sales chart.
- View personal information and reset own username and password.
- Log out safely to protect system data.

As an Employee

- Log in securely to the employee dashboard using authorized credentials.
- View personal information and account details.
- View and search premium customers.
- Add new premium customers during sales transactions.
- Create sales invoices by selecting products and entering quantities.
- View sales reports and print invoices for customers.
- Reset own username and password for security purposes.
- Log out safely after completing work.

Chapter 03a: ER Diagram



ER Diagram of the Farm Invoice Manager database

Explanation of Entities and Relationships:

UserInfo (stores admin and employee information)

- USER_ID (Primary Key, A-xxx for admin, E-xxx for employee)
- ROLE (admin / employee)

- FULL_NAME, USERNAME, EMAIL, MOBILE, PASSWORD, NID, ADDRESS
- CREATED_DATE
- STATUS (1 = Active, 0 = Inactive)

CustomerInfo (stores premium customer details)

- CUSTOMER_ID (Primary Key, PC-xxx)
- CUSTOMER_NAME
- CUSTOMER_MOBILE

ProductInfo (stores product and inventory details)

- PRODUCT_ID (Primary Key, FIM-xxx)
- PRODUCT_NAME, QUANTITY (stock), CATEGORY_NAME, UNITPRICE
- CREATED_DATE, IS_AVAILABLE
- CREATED_BY_USER_ID (Foreign Key $\rightarrow$ UserInfo)

InvoiceDetails (stores the summary of each invoice)

- INVOICE_ID (Primary Key, INV-xxx)
- USER_ID (Foreign Key $\rightarrow$ UserInfo, the user who made the sale)
- CUSTOMER_ID (PC-xxx for a premium customer, RC-xxx for a regular customer)
- CUSTOMER_NAME, CUSTOMER_MOBILE (stored on the invoice for quick display)
- TOTAL_AMOUNT, DISCOUNT_AMOUNT, FINAL_AMOUNT
- INVOICE_DATE

InvoiceItems (stores the products of each invoice)

- INVOICE_ITEM_ID (Primary Key, auto number)
- INVOICE_ID (refers to InvoiceDetails)
- PRODUCT_ID (Foreign Key $\rightarrow$ ProductInfo)
- PRODUCT_NAME, QUANTITY, UNIT_PRICE, SUB_TOTAL

Relationships

- One **User** creates many **Products** (1 : N).
- One **User** makes many **Invoices** (1 : N).
- One **Invoice** has many **Invoice Items** (1 : N).
- One **Product** can be sold in many **Invoice Items** (1 : N).
- One premium **Customer** can buy many times, so one customer has many **Invoices** (1 : N). Regular customers are not saved in CustomerInfo, so not every invoice has a customer record.

Chapter 03b: SQL Queries

1. Database Creation

The database stores all information about users, customers, products, invoices and sales.

Database Name: FarmInvoiceManager. The complete script (database, tables and sample data) is `Database/FarmInvoiceManager.sql`.

```
CREATE DATABASE FarmInvoiceManager;
GO
USE FarmInvoiceManager;
```

2. Table Creation

Five tables represent the main entities of the system:

- **UserInfo** – stores admin and employee information
- **CustomerInfo** – stores premium customer details
- **ProductInfo** – stores product information
- **InvoiceDetails** – stores invoice summary data
- **InvoiceItems** – stores item-wise invoice details

```
CREATE TABLE UserInfo (
    USER_ID NVARCHAR(10) NOT NULL PRIMARY KEY,
    ROLE NVARCHAR(10) NOT NULL,
    FULL_NAME NVARCHAR(50) NOT NULL,
    USERNAME NVARCHAR(50) NOT NULL,
    EMAIL NVARCHAR(50) NOT NULL,
    MOBILE NVARCHAR(20) NOT NULL,
    PASSWORD NVARCHAR(50) NOT NULL,
    NID NVARCHAR(20) NOT NULL,
    ADDRESS NVARCHAR(50) NOT NULL,
    CREATED_DATE DATE NOT NULL,
    STATUS BIT NOT NULL
);

CREATE TABLE CustomerInfo (
    CUSTOMER_ID NVARCHAR(10) NOT NULL PRIMARY KEY,
    CUSTOMER_NAME NVARCHAR(50) NOT NULL,
    CUSTOMER_MOBILE NVARCHAR(20) NOT NULL
);

CREATE TABLE ProductInfo (
    PRODUCT_ID NVARCHAR(10) NOT NULL PRIMARY KEY,
    PRODUCT_NAME NVARCHAR(50) NOT NULL,
    QUANTITY INT NOT NULL,
    CATEGORY_NAME NVARCHAR(50) NOT NULL,
    UNITPRICE FLOAT NOT NULL,
    CREATED_DATE DATE NOT NULL,
    IS_AVAILABLE BIT NOT NULL,
    CREATED_BY_USER_ID NVARCHAR(10) NOT NULL REFERENCES UserInfo(USER_ID)
);

CREATE TABLE InvoiceDetails (
    INVOICE_ID NVARCHAR(10) NOT NULL PRIMARY KEY,
    USER_ID NVARCHAR(10) NOT NULL REFERENCES UserInfo(USER_ID),
    CUSTOMER_ID NVARCHAR(10) NOT NULL,
    CUSTOMER_NAME NVARCHAR(50) NOT NULL,
    CUSTOMER_MOBILE NVARCHAR(20) NOT NULL,
    TOTAL_AMOUNT DECIMAL(10,2) NOT NULL,
    DISCOUNT_AMOUNT DECIMAL(10,2) NOT NULL,
    FINAL_AMOUNT DECIMAL(10,2) NOT NULL,
    INVOICE_DATE DATE NOT NULL
);

CREATE TABLE InvoiceItems (
    INVOICE_ITEM_ID INT IDENTITY(1,1) PRIMARY KEY,
    INVOICE_ID NVARCHAR(10) NOT NULL,
    PRODUCT_ID NVARCHAR(10) NOT NULL REFERENCES ProductInfo(PRODUCT_ID),
    PRODUCT_NAME NVARCHAR(100) NOT NULL,
```

```
    QUANTITY INT NOT NULL,
    UNIT_PRICE DECIMAL(10,2) NOT NULL,
    SUB_TOTAL DECIMAL(10,2) NOT NULL
);
```

3. Queries Used in the Application

All queries are sent with parameters (`@Name`) through one database class (`Data/Db.cs`), and the connection string is written only once in `App.config`. This protects the system from SQL injection. The main queries of each form are given below.

FormLogin

```
SELECT USER_ID, FULL_NAME, ROLE, STATUS FROM UserInfo
WHERE (USERNAME = @User OR USER_ID = @User) AND PASSWORD = @Password;
```

FormFindPassword

```
SELECT PASSWORD FROM UserInfo
WHERE USER_ID = @ID AND USERNAME = @Username AND MOBILE = @Mobile;
```

FormAdminView

```
SELECT * FROM UserInfo ORDER BY USER_ID;

SELECT * FROM UserInfo
WHERE FULL_NAME LIKE @Search OR USERNAME LIKE @Search OR USER_ID LIKE @Search
ORDER BY USER_ID;

DELETE FROM UserInfo WHERE USER_ID = @ID;
DELETE FROM CustomerInfo WHERE CUSTOMER_ID = @ID;
```

FormAdminView and FormEmployee (shared: customers and personal information)

```
SELECT * FROM CustomerInfo ORDER BY CUSTOMER_ID;

SELECT * FROM CustomerInfo WHERE CUSTOMER_MOBILE LIKE @Search ORDER BY CUSTOMER_ID;

SELECT * FROM UserInfo WHERE USER_ID = @ID;
```

FormAddUser

```
-- next ID: admins (A-) and employees (E-) share one counter
SELECT ISNULL(MAX(CAST(SUBSTRING(USER_ID, 3, 10) AS INT)), 0)
FROM UserInfo WHERE USER_ID LIKE '[AE]-%';

SELECT COUNT(*) FROM UserInfo WHERE USERNAME = @U;

INSERT INTO UserInfo (USER_ID, ROLE, FULL_NAME, USERNAME, EMAIL, MOBILE,
                      PASSWORD, NID, ADDRESS, CREATED_DATE, STATUS)
VALUES (@ID, @Role, @FullName, @Username, @Email, @Mobile,
        @Password, @NID, @Address, CAST(GETDATE() AS DATE), @Status);
```

FormEditUser

```
SELECT * FROM UserInfo WHERE USER_ID = @ID;
```

```
SELECT COUNT(*) FROM UserInfo WHERE USERNAME = @U AND USER_ID <> @ID;

UPDATE UserInfo
SET FULL_NAME = @FullName, EMAIL = @Email, USERNAME = @Username,
    PASSWORD = @Password, MOBILE = @Mobile, NID = @NID,
    ADDRESS = @Address, STATUS = @Status
WHERE USER_ID = @ID;
```

FormReset

```
SELECT USERNAME, PASSWORD FROM UserInfo WHERE USER_ID = @ID;

UPDATE UserInfo SET USERNAME = @NewUser, PASSWORD = @NewPassword
WHERE USER_ID = @ID AND USERNAME = @OldUser AND PASSWORD = @OldPassword;
```

FormManageProducts

```
SELECT * FROM ProductInfo ORDER BY PRODUCT_ID;

SELECT * FROM ProductInfo
WHERE PRODUCT_NAME LIKE @Search OR PRODUCT_ID LIKE @Search
ORDER BY PRODUCT_ID;

DELETE FROM ProductInfo WHERE PRODUCT_ID = @ID;
```

FormAddProduct

```
SELECT ISNULL(MAX(CAST(SUBSTRING(PRODUCT_ID, 5, 10) AS INT)), 0)
FROM ProductInfo WHERE PRODUCT_ID LIKE 'FIM-%';

INSERT INTO ProductInfo (PRODUCT_ID, PRODUCT_NAME, QUANTITY, CATEGORY_NAME,
                         UNITPRICE, CREATED_DATE, IS_AVAILABLE, CREATED_BY_USER_ID)
VALUES (@ID, @Name, @Qty, @Category, @Price, @Date, @Available, @CreatedBy);
```

FormUpdateProduct

```
SELECT * FROM ProductInfo WHERE PRODUCT_ID = @ID;

UPDATE ProductInfo
SET PRODUCT_NAME = @Name, QUANTITY = @Qty, CATEGORY_NAME = @Category,
    UNITPRICE = @Price, CREATED_DATE = @Date, IS_AVAILABLE = @Available
WHERE PRODUCT_ID = @ID;
```

FormAddCustomer

```
SELECT COUNT(*) FROM CustomerInfo WHERE CUSTOMER_MOBILE = @Mobile;

INSERT INTO CustomerInfo (CUSTOMER_ID, CUSTOMER_NAME, CUSTOMER_MOBILE)
VALUES (@ID, @Name, @Mobile);
```

FormAddSales

```
SELECT PRODUCT_ID, PRODUCT_NAME FROM ProductInfo
WHERE IS_AVAILABLE = 1 AND QUANTITY > 0 ORDER BY PRODUCT_NAME;

SELECT CUSTOMER_ID, CUSTOMER_MOBILE FROM CustomerInfo ORDER BY CUSTOMER_MOBILE;
SELECT CUSTOMER_NAME FROM CustomerInfo WHERE CUSTOMER_MOBILE = @Mobile;
SELECT UNITPRICE, QUANTITY FROM ProductInfo WHERE PRODUCT_ID = @ID;
```

```
-- saved together in one transaction
INSERT INTO InvoiceDetails (INVOICE_ID, USER_ID, CUSTOMER_ID, CUSTOMER_NAME,
          CUSTOMER_MOBILE, TOTAL_AMOUNT, DISCOUNT_AMOUNT, FINAL_AMOUNT, INVOICE_DATE)
VALUES (@Invoice, @User, @Customer, @Name, @Mobile, @Total, @Discount, @Final, @Date);

INSERT INTO InvoiceItems (INVOICE_ID, PRODUCT_ID, PRODUCT_NAME, QUANTITY, UNIT_PRICE,
    SUB_TOTAL)
VALUES (@Invoice, @Product, @Name, @Qty, @Price, @SubTotal);

UPDATE ProductInfo SET QUANTITY = QUANTITY - @Qty WHERE PRODUCT_ID = @Product;
```

FormInvoicePrint

```
SELECT CUSTOMER_NAME, CUSTOMER_MOBILE, TOTAL_AMOUNT, DISCOUNT_AMOUNT,
      FINAL_AMOUNT, INVOICE_DATE
FROM InvoiceDetails WHERE INVOICE_ID = @ID;

SELECT PRODUCT_NAME, QUANTITY, UNIT_PRICE, SUB_TOTAL
FROM InvoiceItems WHERE INVOICE_ID = @ID ORDER BY INVOICE_ITEM_ID;
```

FromSalesReport

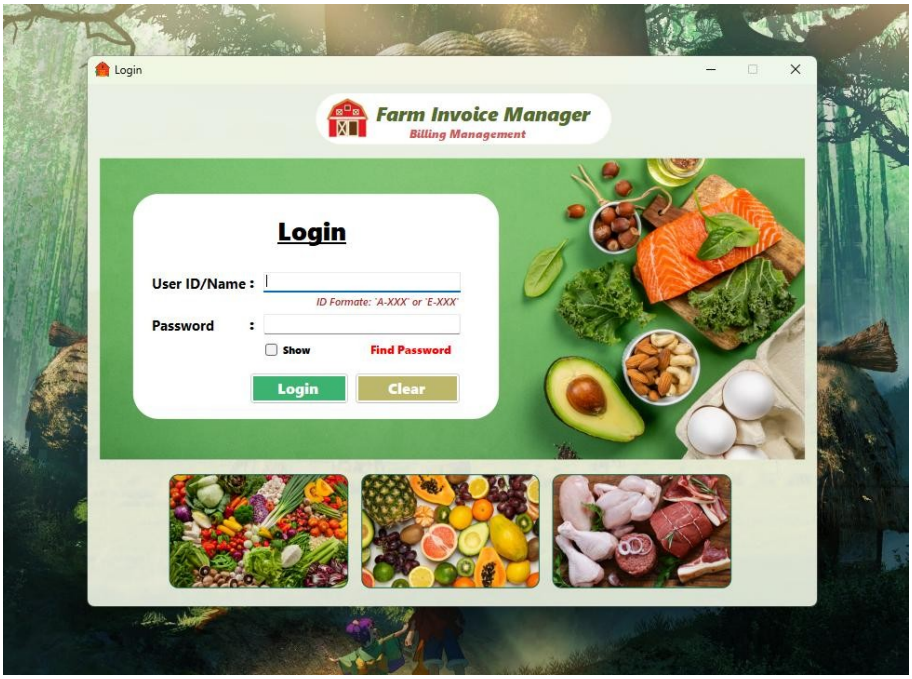
```
-- the date and mobile filters are added only when they are used
SELECT * FROM InvoiceDetails
WHERE INVOICE_DATE BETWEEN @From AND @To
  AND CUSTOMER_MOBILE LIKE @Mobile
ORDER BY INVOICE_DATE, INVOICE_ID;

SELECT PRODUCT_ID, PRODUCT_NAME, QUANTITY, UNIT_PRICE, SUB_TOTAL
FROM InvoiceItems WHERE INVOICE_ID = @ID;

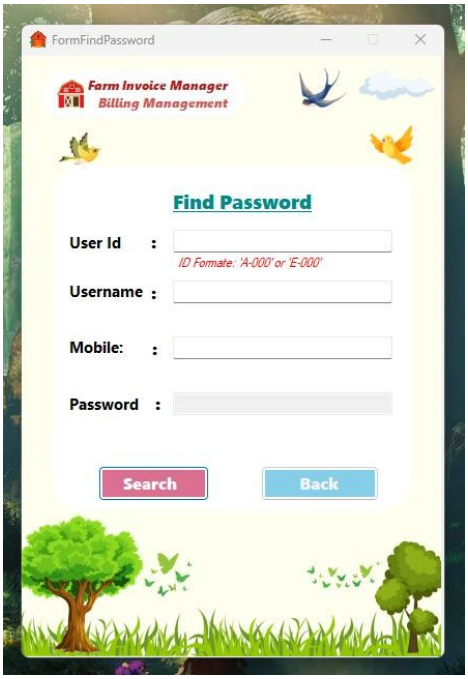
-- deleted together in one transaction
DELETE FROM InvoiceItems WHERE INVOICE_ID = @ID;
DELETE FROM InvoiceDetails WHERE INVOICE_ID = @ID;
```

Chapter 04: Screenshots

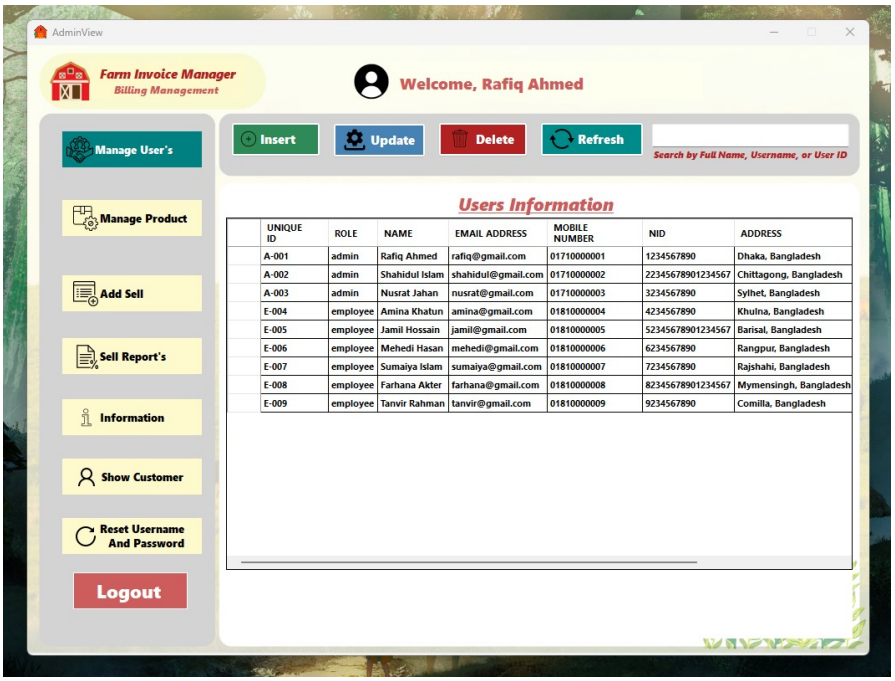
Form Name: Login Form
Purpose: User authentication (Admin / Employee).



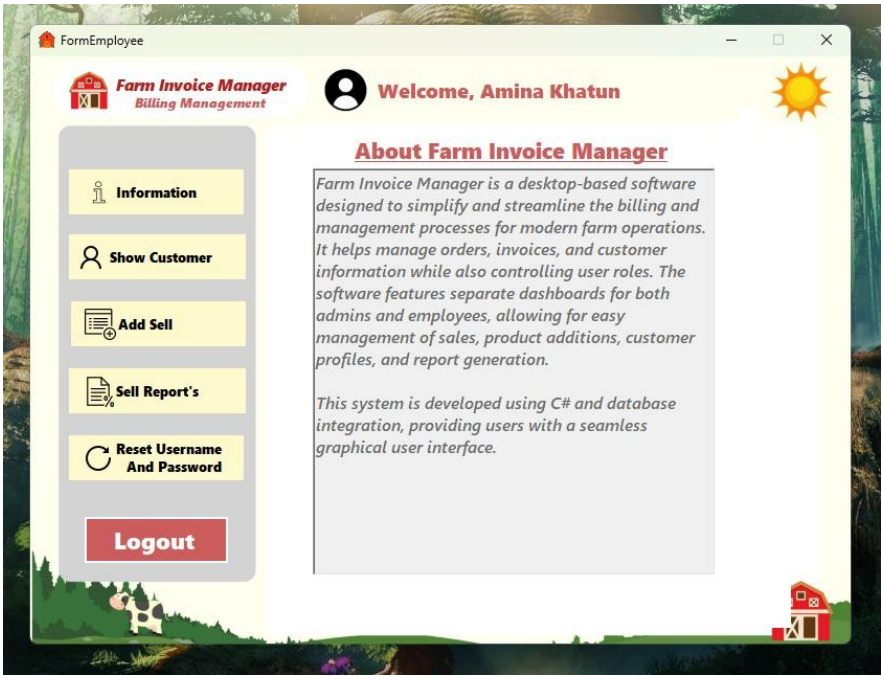
Form Name: Find Password
Purpose: Recover a forgotten password with User ID, username and mobile number.



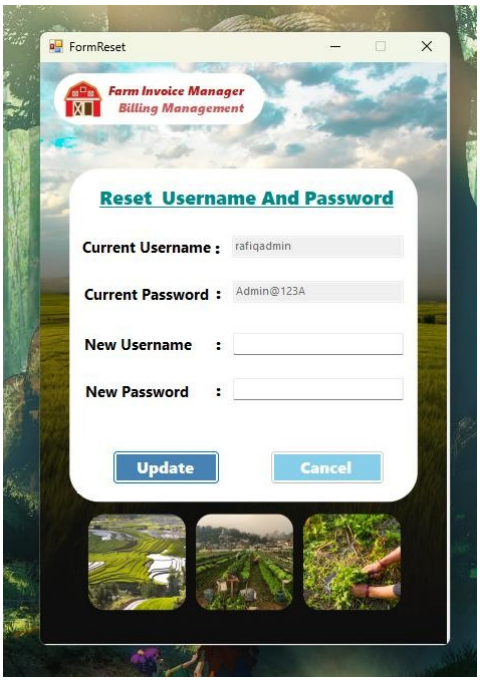
Form Name: Admin Dashboard
Purpose: Admin main control panel for managing users.



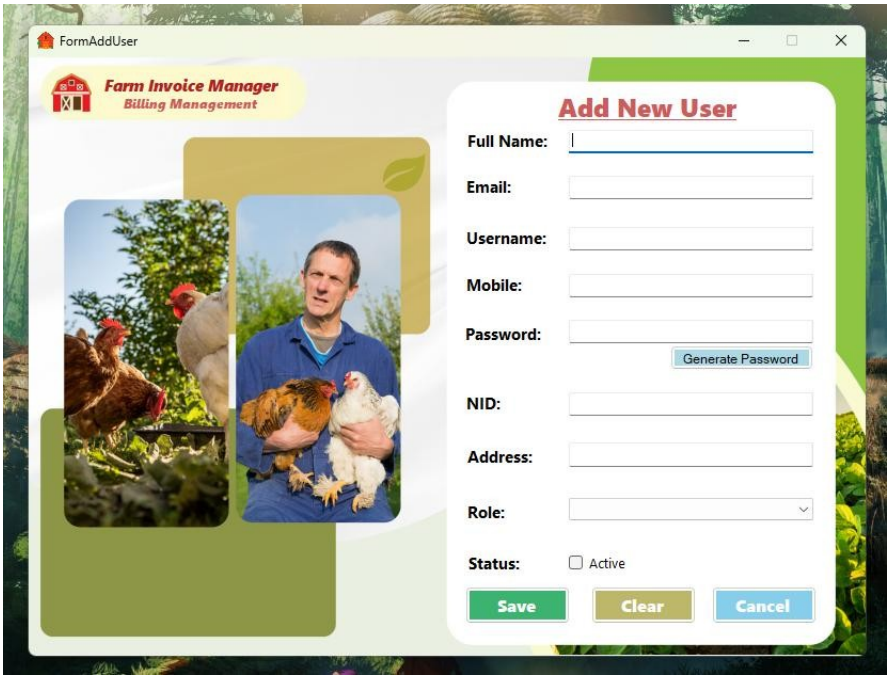
Form Name: Employee Dashboard
Purpose: Employee main control panel.



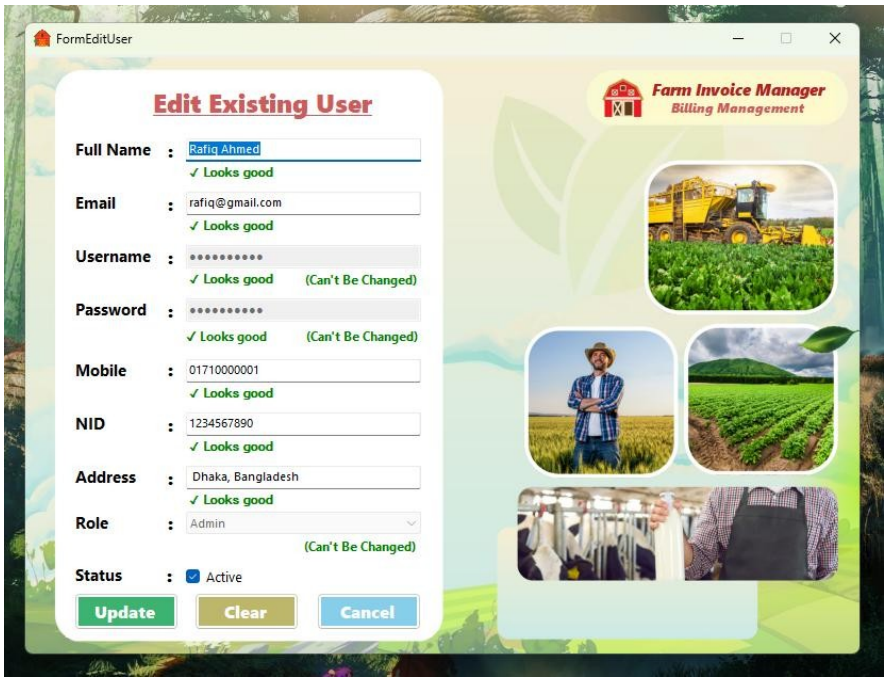
Form Name: Reset Username and Password Form
Purpose: Change the username and password of the logged-in user.



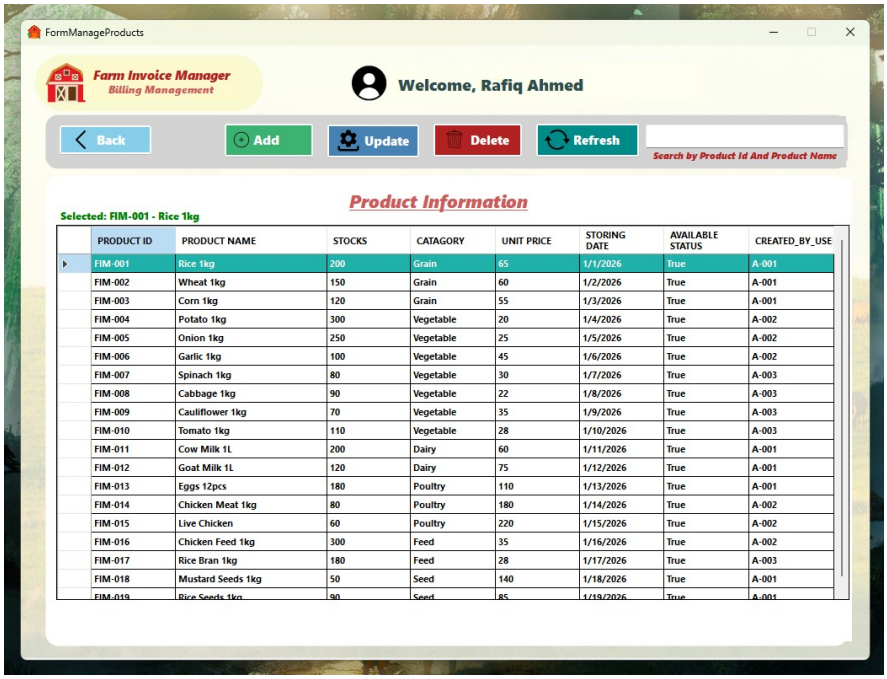
Form Name: Add New User Form
Purpose: Create an admin or employee account.



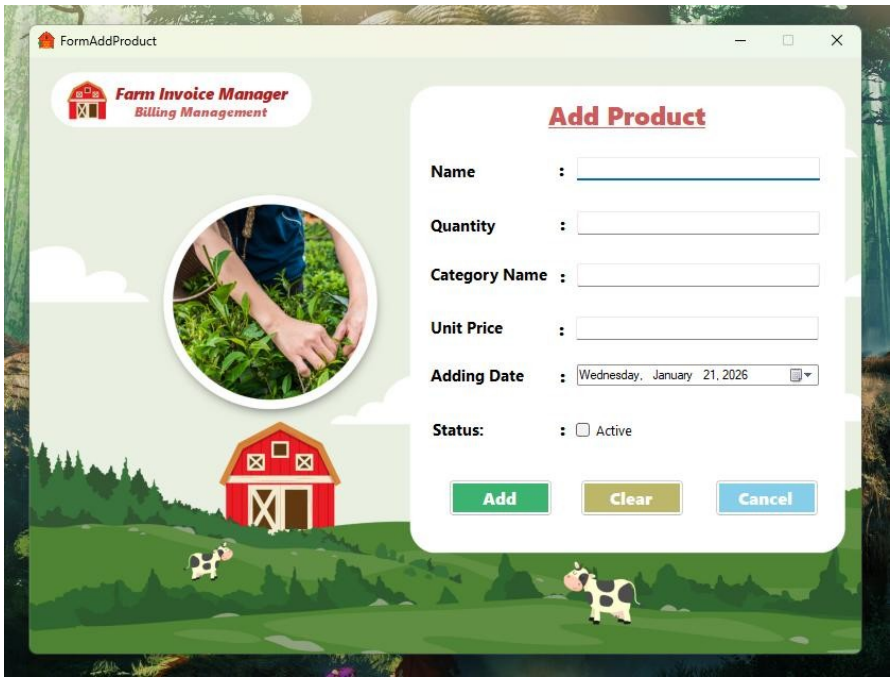
Form Name: Edit Existing User Form
Purpose: Update user information.



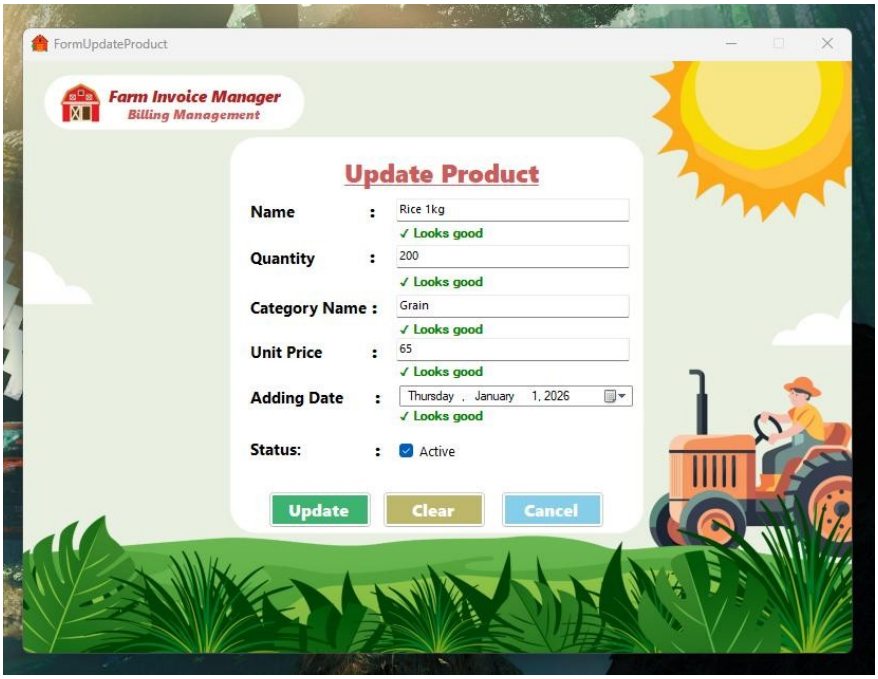
Form Name: Manage Products Form
Purpose: View, search, add, update and delete products.



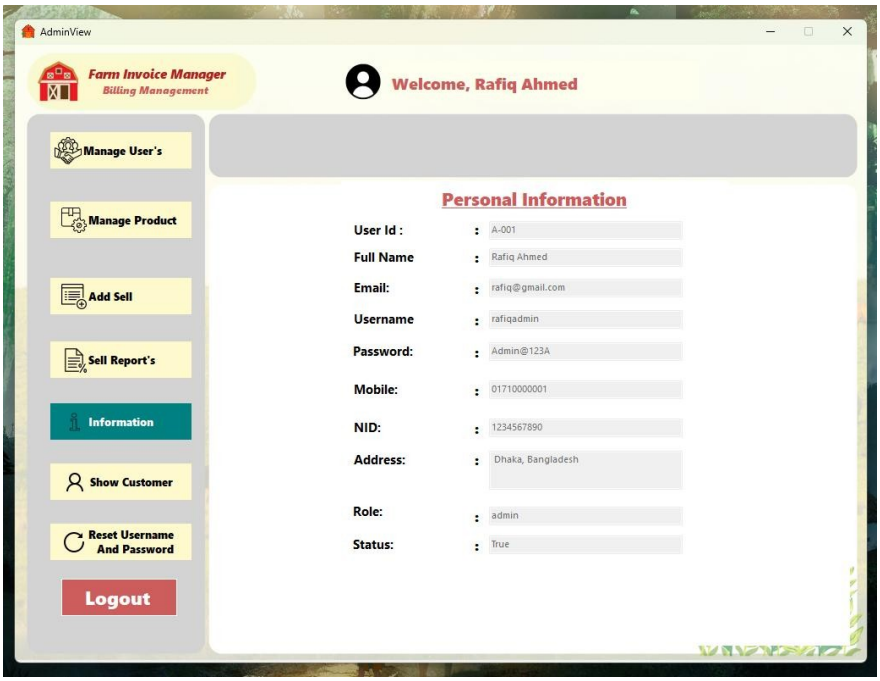
Form Name: Add Product Form
Purpose: Add new product information.



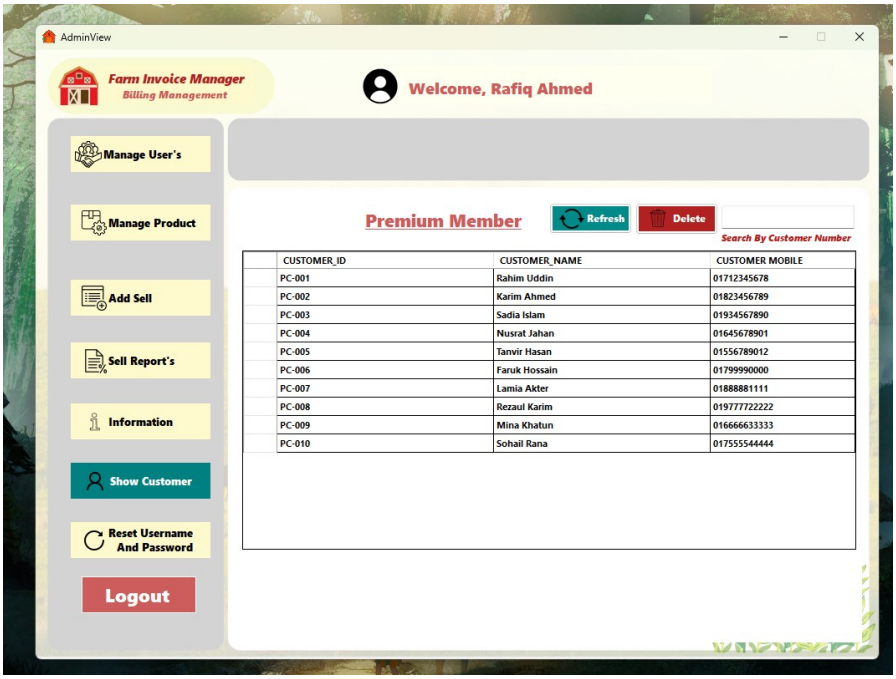
Form Name: Update Product Form
Purpose: Update existing product details.



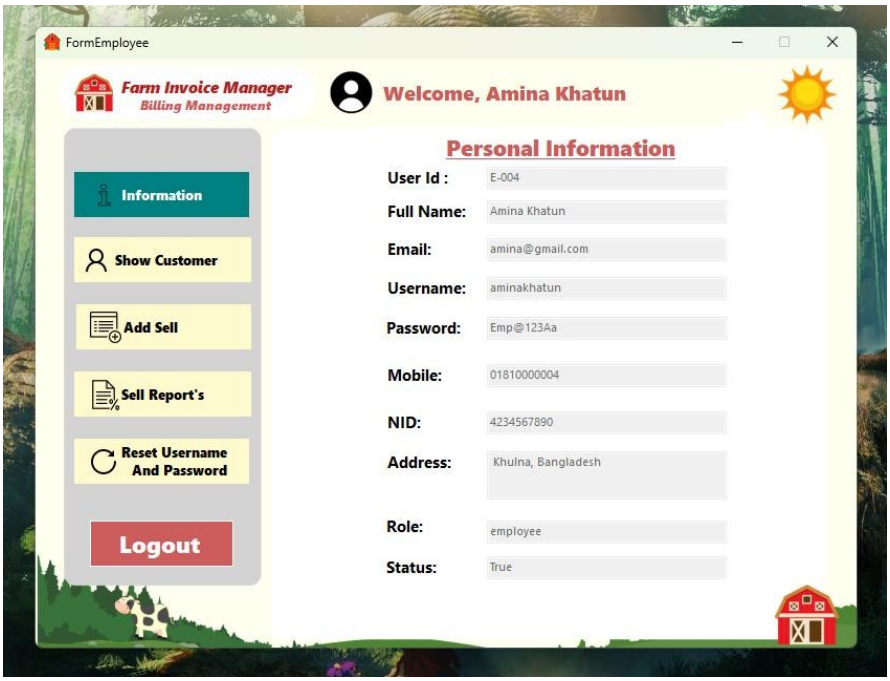
Form Name: Personal Information Form (Admin)
Purpose: View logged-in admin details.



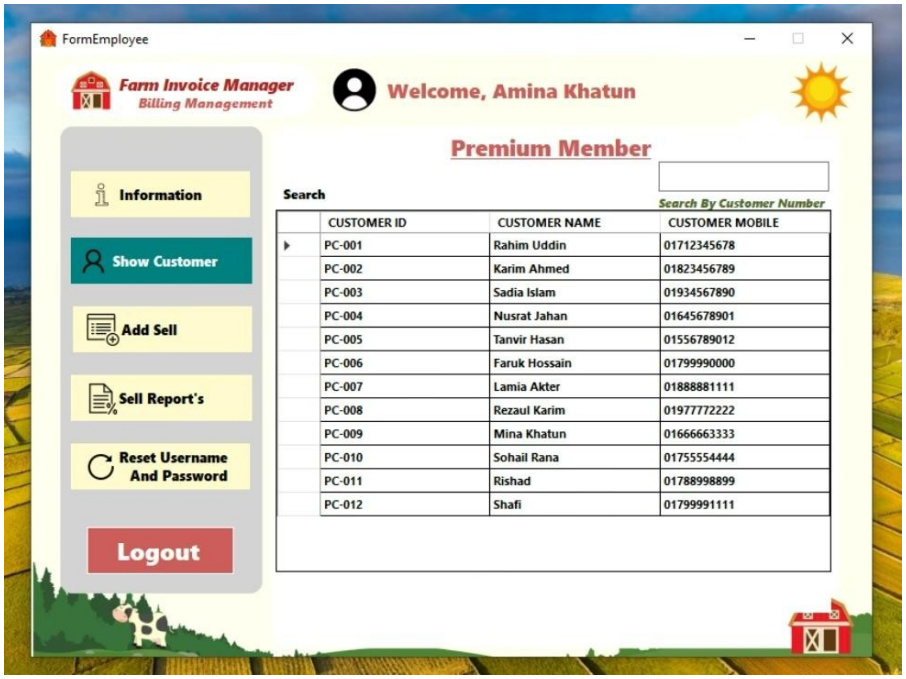
Form Name: Show Customer Form (Admin)
Purpose: View, search and delete premium customers.



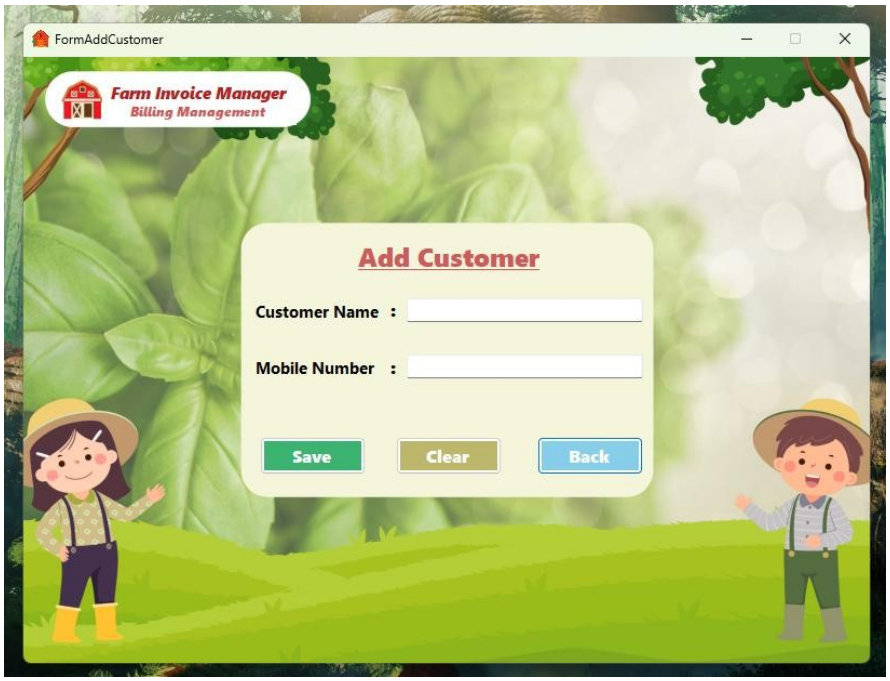
Form Name: Personal Information Form (Employee)
Purpose: View logged-in employee details.



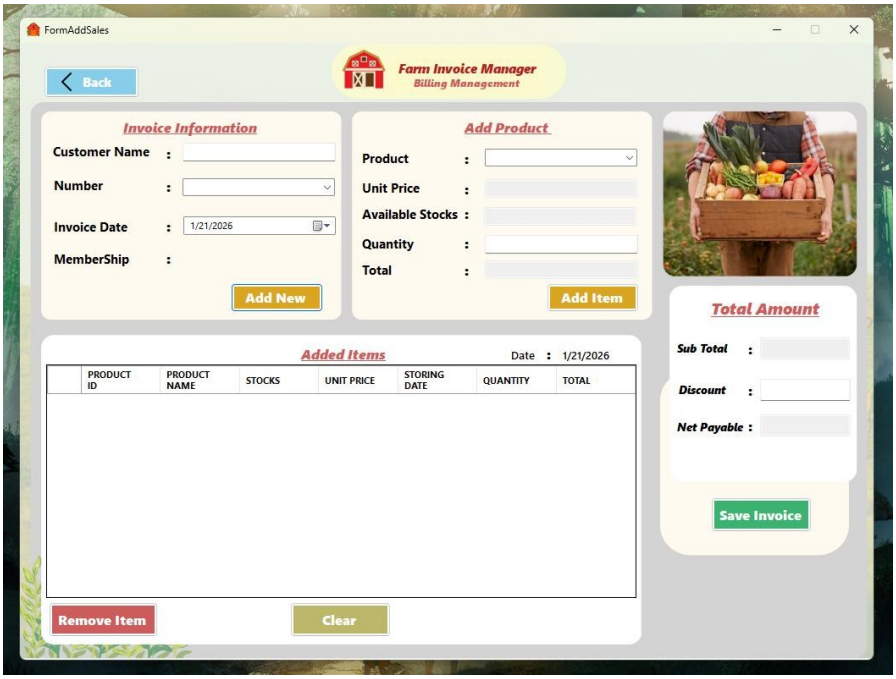
Form Name: Show Customer Form (Employee)
Purpose: View and search premium customers.



Form Name: Add Customer Form
Purpose: Add a new premium customer.



Form Name: Add Sales Form
Purpose: Create and save customer sales invoices.



Form Name: Sales Report Form
Purpose: View and analyze date-wise sales information.

